

Kivaa: Aplicación Móvil Para Celíacos

Andrea Sofía Pais Dos Santos

June 13, 2026

Índice

1	Introducción	3
1.1	Contexto Actual y Problema	3
1.2	Definición de la Propuesta	4
1.3	Objetivos del Proyecto	4
1.3.1	Objetivo General	4
1.3.2	Objetivos Específicos	4
2	Análisis de Requisitos	4
2.1	Requisitos Funcionales	4
2.2	Requisitos No Funcionales	5
2.3	Historias de Usuario	5
2.4	Diagrama de Casos de Uso	6
2.5	Esquema Entidad - Relación	8
2.6	Diagrama de Clases	8
3	Tecnologías Utilizadas	9
3.1	Frontend : Móvil	9
3.2	Backend	10
3.3	Base de Datos	10
3.4	Herramientas Auxiliares	10
4	Diseño del Sistema	11
4.1	Arquitectura de la App Cliente-Servidor	11
4.2	Diseño de la Base de Datos	11
4.3	Diseño de la API REST	11
4.3.1	Seguridad (JWT, validación Zod y bcrypt)	13
4.4	Creación de la Marca y Prototipado de la Interfaz (UX/UI)	16
5	Desarrollo de la Aplicación Móvil	20
5.1	Implementación del Frontend	20
5.2	Integración con la API y Base de Datos	26
6	Estudio de Viabilidad y Utilización Empresarial	27
7	Pruebas y Validación	27
7.1	Pruebas de Usuario	27
8	Escalabilidad de Kivaa	29
9	Manual de Instalación y Configuración	29
10	Conclusiones	31
11	Bibliografía	32

1 Introducción

1.1 Contexto Actual y Problema

Actualmente, la sociedad empieza a entender el verdadero problema que le supone a un celíaco o a una persona intolerante buscar sitios para comer o comprar productos sin gluten. Ser celíaco o intolerante no es una elección, sino una necesidad de salud que afecta al **1% de la población aproximadamente** más otro porcentaje que ni está diagnosticado. [1]

Por una parte, aunque la industria alimentaria haya aumentado la producción de productos sin gluten siguen manteniendo un coste un 300% más caro que un producto convencional y aún así sigue siendo limitados y poco variados.

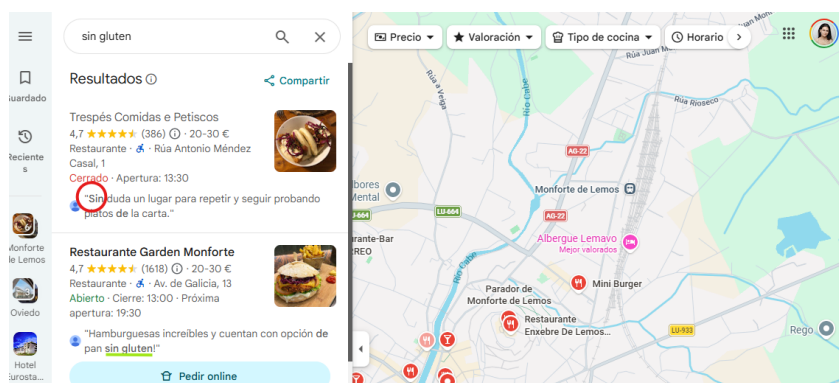


Por otra, la búsqueda de establecimientos que sean seguros para evitar contaminaciones cruzadas son pocos y difícil de encontrar. Normalmente, las personas que padecen de esta enfermedad, **suelen conocer nuevos sitios gracias a su círculo cercano o por lo que vemos en redes**. Pero los propios establecimientos no actualizan sus cartas ni sus protocolos sanitarios para evitar las contaminaciones cruzadas.

Esto aumenta al viajar a otras zonas, que acaba limitando los sitios a los que puede ir por falta de conocimiento al no ser capaz de encontrar un lugar de manera intuitiva.

Para un celíaco es muy complicado poder ir a comer fuera por miedo a que se equivoquen de alimentos, o ya no dispongan de una alternativa para ellos. **La sociedad cada vez más empatiza con este colectivo pero falta que a nivel tecnológico se pueda crear un puente entre los establecimientos que ofrecen opciones seguras y los usuarios** que las necesita para poder vivir algo más tranquilo.

Al buscar en Google Maps para comprobar la incertidumbre que tienen los celíacos, en el buscador al poner "sin gluten" lo que hace es una búsqueda en las reseñas y coge a veces solo la palabra "sin" pero no hay nada que demuestre que ese sitio tiene opción si gluten. Está muy bien que en las cartas pongan el icono que contiene gluten pero si no aportan una alternativa sigue sin solucionar nada.



1.2 Definición de la Propuesta

La propuesta del proyecto se basa en intentar **satisfacer una necesidad que se puede cubrir creando una aplicación para móvil** donde los usuarios puedan buscar establecimientos cerca de su ubicación o en otros lugares que dispongan de menú alternativo sin gluten.

Para confirmar la veracidad de los establecimientos, otros usuarios que padecen de la misma enfermedad comentarán su experiencia y le dará más credibilidad a los lugares a los que otros usuarios no hayan ido.

1.3 Objetivos del Proyecto

1.3.1 Objetivo General

El objetivo principal del proyecto es **diseñar y desarrollar una solución al problema plantado anteriormente mediante una aplicación móvil que garantice a los usuarios que están buscando sitios sin gluten** recomendados por otras personas que también son celíacas o intolerantes.

1.3.2 Objetivos Específicos

- Conseguir que ya no sea necesario ese círculo cercano para obtener información y haya un inventario digital muy completo actualizando los locales que vayan apareciendo sin gluten.
- Conseguir esa fiabilidad mediante las experiencias reales de los usuarios con las valoraciones y comentarios.
- Intentar facilitar la búsqueda de los establecimientos en tiempo real mediante el uso de mapas interactivos.
- Desarrollar una única base con React Native y Expo para asegurar la compatibilidad en Android/iOS.
- Diseñar una arquitectura de base de datos no relacional en MongoDB que vaya a almacenar información escalable y flexible para futuras mejoras.
- Implementar el SDK de Google Maps para la visualización y filtrado por ubicación del usuario.
- Diseñar una interfaz intuitiva y accesible en la que puedas encontrar un establecimiento en 5 minutos.

2 Análisis de Requisitos

2.1 Requisitos Funcionales

Los usuarios van a poder realizar las siguientes acciones dependiendo de que quiera hacer:

Gestionar el Perfil y Usuario

- **RF1 - Inicio de Sesión y Registro:** el usuario tiene que poder crear una cuenta de forma intuitiva y poder iniciar sesión con la misma enseñando validaciones para que el usuario pueda ver si tiene un error o está todo correcto.
- **RF2 - Gestionar su Perfil:** el usuario tiene que poder editar sus datos personales si le interesa o cerrar sesión.

Buscar Establecimientos y Localización

- **RF3 - Localización en tiempo real:** la aplicación tiene que cargar la ubicación del usuario a través del GPS del móvil.
- **RF4 - Ver en Mapa:** poder ver los establecimientos cercanos sin gluten a través de Google Maps.
- **RF5 - Buscador con Filtros:** el buscador tiene que poder filtrar por nombre del establecimiento, lugar y valoraciones.

Información Adicional de los Establecimientos

- **RF6 - Fichas de establecimientos:** El usuario al darle a un establecimiento puede ver la Información detallada del mismo, con sus fotos, dirección, teléfono y comentarios de otros usuarios.
- **RF7 - Creación de Reseñas y Valoraciones:** los usuarios pueden valorar los establecimientos a través de una puntuación de 1 a 5 estrellas y añadir un comentario.

Administración de lugares

RF8 - Creación de los establecimientos: un administrador tiene que poder añadir nuevos establecimientos a la base de datos de MongoDB.

2.2 Requisitos No Funcionales

Estos requisitos son los que definen la calidad del sistema:

RNF1 - Seguridad: las contraseñas en la base de datos deberían estar encriptadas.

RNF2 - Usabilidad: la interfaz tiene que ser intuitiva para que el usuario se sienta cómodo al usar la aplicación siguiendo unas bases de diseño funcional.

RNF3 - Rendimiento: las consultas que realicen los usuarios a la API Express no puede superar un tiempo de respuesta considerado largo.

2.3 Historias de Usuario

Bloque Administrador

ID	Historias de Uso	Criterios de Aceptación
HU-06	Como administrador tengo que borrar un establecimiento porque ya no posee una alternativa sin gluten.	1. Seleccionar el local. 2. Borrar a través del botón y doble confirmación.
HU-07	Como administrador, quiero validar un nuevo establecimiento para asegurar que la información de la app es verídica.	1. El admin recibe una notificación de nuevo local. 2. Puede editar los datos del local antes de subir. 3. Al confirmar, el local aparece para todos.
HU-08	Como administrador tengo que borrar los comentarios inapropiados.	1. Seleccionar el local. 2. Seleccionar la reseña inapropiada. 3. Borrar a través del botón y la doble confirmación.

Bloque Búsqueda

ID	Historias de Uso	Criterios de Aceptación
HU-03	Como usuario celiaco, quiero ver un mapa con locales cercanos para encontrar dónde comer sin desplazarme mucho.	1. El mapa muestra la ubicación actual del usuario. 2. Se ven marcadores de los locales "sin gluten". 3. Al pulsar un marcador, se ve el nombre del local.
HU-04	Como usuario quiero filtrar un establecimiento por el nombre.	1. En el buscador del mapa, escribir el nombre del local. 2. Darle al botón de buscar. 3. Marcadores de los sitios con ese mismo nombre.
HU-05	Como usuario quiero ver una ficha detallada de un local para contactar con él.	1. Buscar un local. 2. Seleccionarlo y se muestra la ficha del local.

Bloque Usuarios

ID	Historias de Uso	Criterios de Aceptación
HU-01	Como usuario necesito crear una cuenta para acceder a la aplicación.	1. Usuario rellena formulario de registro. 2. Crear la cuenta. 3. Ahora puede iniciar sesión.
HU-02	Como usuario tengo que poder entrar con la cuenta que he creado.	1. El usuario rellena el formulario con el correo y contraseña. 2. Accede a la app si las credenciales son correctas.

Bloque Reseñas

ID	Historias de Uso	Criterios de Aceptación
HU-09	Como usuario quiero ver las reseñas de un local en específico.	1. Se busca un local y se selecciona. 2. Se obtiene la información del local más las reseñas publicadas.
HU-10	Como usuario quiero subir una reseña de un local en el que estuve.	1. Se busca un local y se selecciona. 2. Dar al botón de añadir reseña. 3. Rellenar formulario y enviar.

2.4 Diagrama de Casos de Uso

El diagrama representa las interacciones principales del Usuario con la aplicación y el Administrador gestionando la aplicación. Por un lado, hay Actores en este caso el Usuario y el Administrador están representando los "stickman" a la izquierda. A continuación se describen las relaciones clave para explicar el flujo y las dependencias de las acciones. Las acciones que necesitan realizar necesitan iniciar sesión con una cuenta.

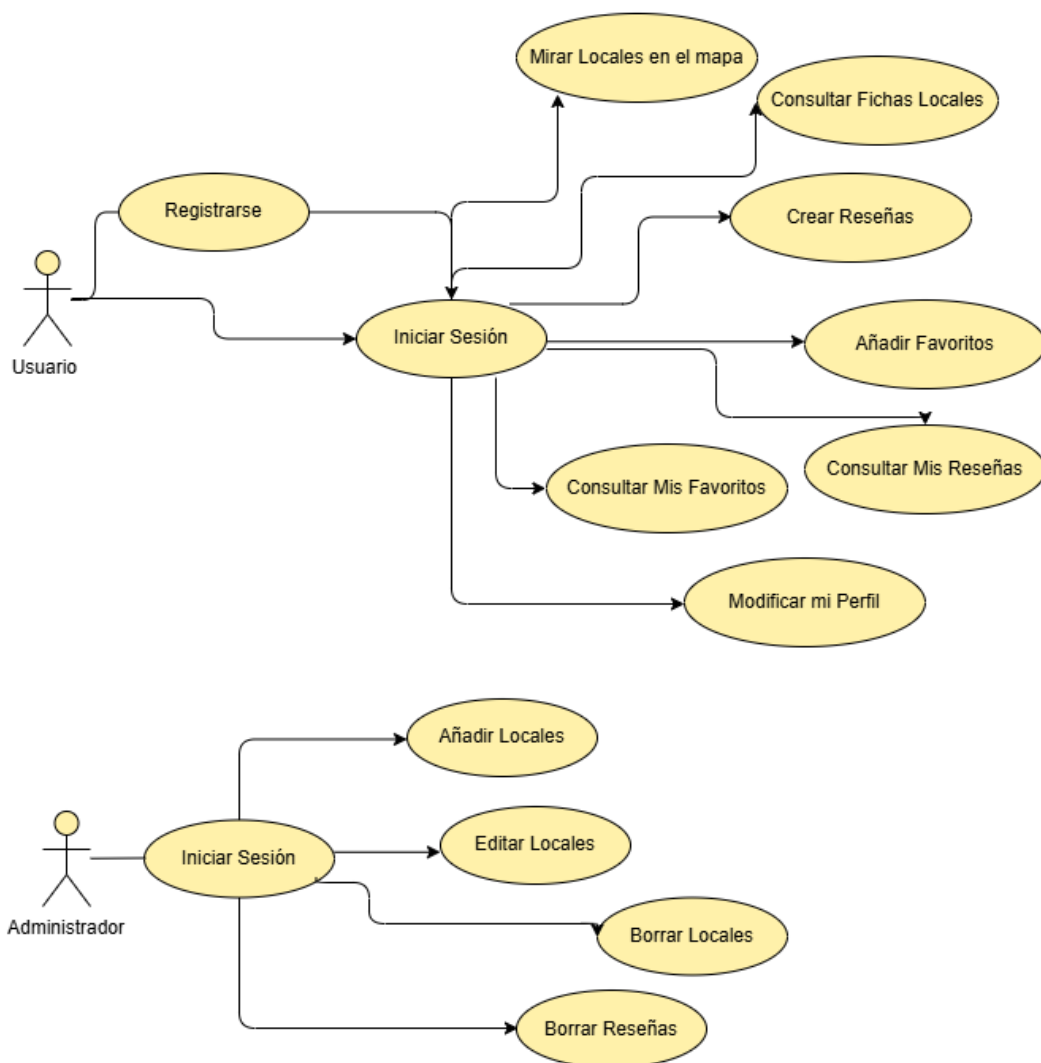
El **Usuario** puede de primeras registrarse si no lo está y luego iniciar sesión o directamente iniciar sesión. Tras tener la cuenta iniciada puede:

- **Buscar un local en el mapa** mediante un buscador que filtra según lo escrito y marca en el mapa el local.
- **Consultar ficha local** luego de la búsqueda anterior o a través del menú de locales seleccionas el local y entras en su información detallada que permite añadir a la lista de favoritos.

- **Crear Reseñas y consultarlas** dentro de la ficha del local se ve la lista de las reseñas y mediante un formulario se puede añadir un comentario junto a una puntuación. Que luego se pueden ver en el apartado del menú principal de la aplicación.
- **Añadir a Favoritos y consultarlos** en al ficha puedes añadir ese local a tu lista de favoritos y verlos en un apartado del menú principal.

El **Administrador** luego de iniciar sesión con su usuario dado puede realizar las siguientes operaciones:

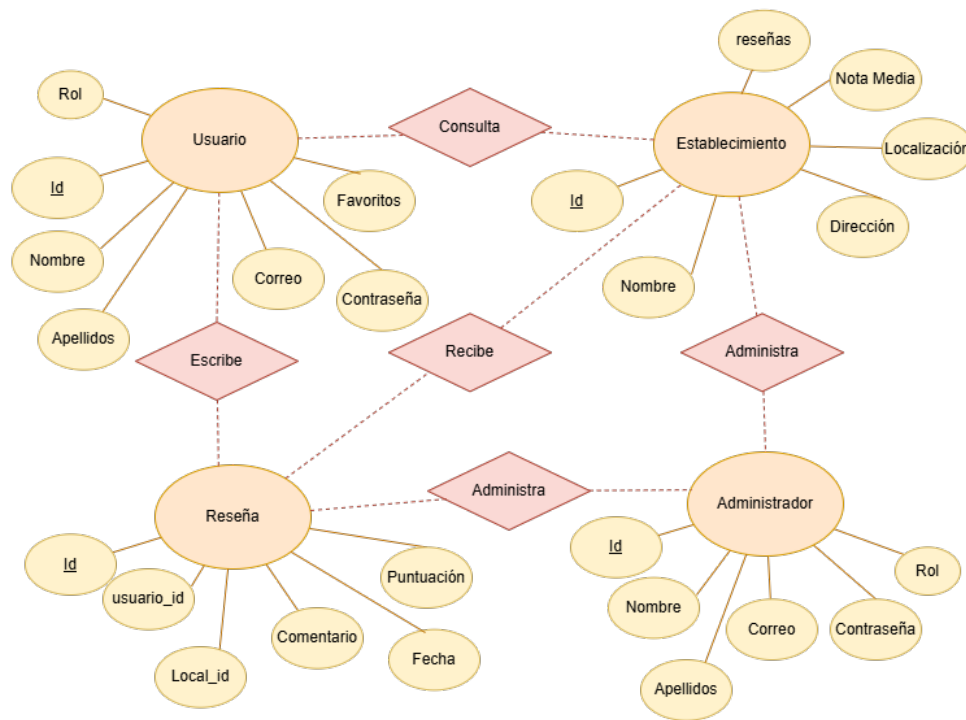
- **Añadir/Editar/Borrar un local** mediante unos formularios el administrador puede realizar esas acciones según lo que interese y guardar los cambios en ese momento.
- **Borrar Comentarios** que se consideren innapropiados o fuera de lugar a través de un botón y boble confirmación.



2.5 Esquema Entidad - Relación

Se ha elaborado el modelo Entidad-Relación para represetar la estructura lógica de los datos de la aplicación. En el modelo se identifican las entidades principales (**Usuario**, **Establecimiento**, **Reseña** y **Administración**) y los atributos.

Este esquema sirve de base para la posterior implementación de las colecciones en el sistema de base de datos NoSQL seleccionado (MongoDB).



2.6 Diagrama de Clases

Para organizar la información de la aplicación, se ha diseñado un diagrama de clases que muestra cómo se relacionan los datos principales. Aunque usamos **MongoDB**, este esquema nos ayuda a estructurar el código en el servidor.

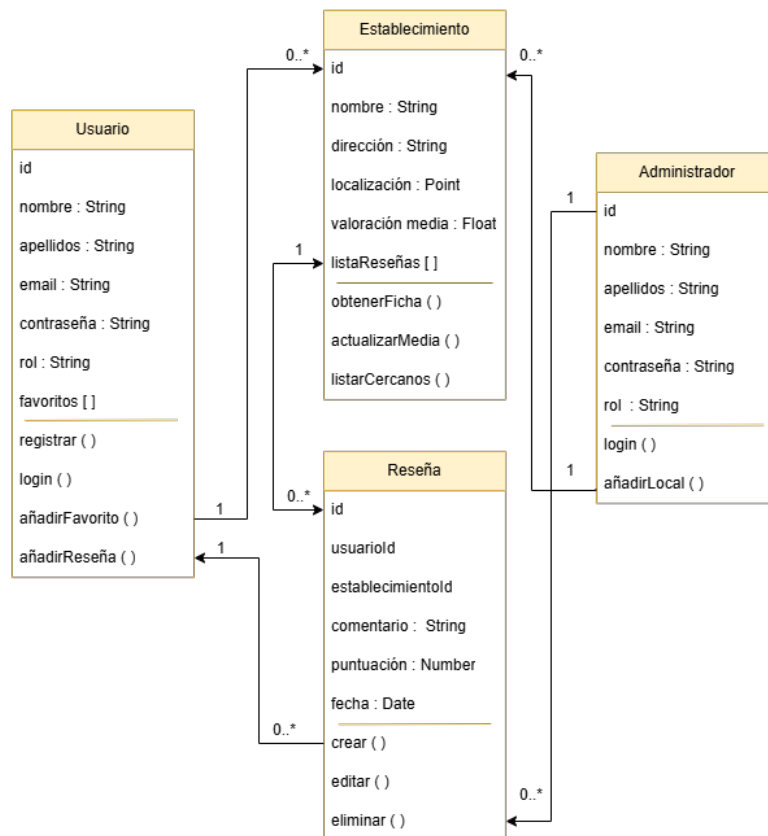
Definición de las clases

- **Usuario:** guarda los datos de las personas que usan la app (atributos) en la que tiene una lista de **favoritos** que permite al usuario guardar sus locales preferidos en una lista y verlos todos juntos.
- **Establecimiento:** tiene una ficha por cada restaurante que se añade y se guarda la **Localización exacta** usando el tipo "Point" para que aparezcan en el mapa, además de la nota media de los usuarios que publiquen una reseña sobre ese establecimiento.
- **Reseña:** es el comentario que escribe un usuario sobre un local vinculando a la persona con el local obteniendo su puntuación y fecha del comentario.
- **Administrador:** es el usuario especial que tiene permitido **añadir, editar o borrar locales o**

borrar comentarios innapropiados para ir manteniendo la aplicación actualizada.

Conexión entre las clases

- **Un usuario puede escribir muchas reseñas**, pero cada reseña es escrita por un único usuario solo.
- **Un local puede recibir muchos comentarios**, pero cada reseña va a un local en específico.
- **Los favoritos** es una relación libre donde muchos usuarios pueden guardar muchos locales diferentes para verlos a parte.



Por lo que este diseño permite que la aplicación sea rápida y que los datos estén siempre bien organizados ayudando a que las personas puedan buscar un sitio sin gluten cerca de su ubicación de forma eficiente.

3 Tecnologías Utilizadas

Para la realización del proyecto en este apartado se definen las herramientas y lenguajes que se van a necesitar dividiendo en 3 capas: Frontend (Móvil), Backend (API) y Persistencia (Base de Datos).

3.1 Frontend : Móvil

Para el desarrollo del Frontend se va a utilizar React Native + Expo.

- **React Native** va a permitir crear la aplicación móvil usando JavaScript y React. Que a diferencia de las aplicaciones web, React Native renderiza componentes nativos por lo que ofrece una experiencia de usuario más fluida. [2]

- **Expo** se compone por un conjunto de herramientas y servicios construidos alrededor de React Native. Esto facilita el acceso al GPS, la cámara y el despliegue. Expo Go o Android Studio para ir viendo en tiempo real los cambios que se van realizando y para los testeos. [3]

3.2 Backend

Para el desarrollo de Backend se va a implementar Node.js + Express. Esta combinación va a permitir crear una API REST eficiente y robusta.[4] Se escogieron porque así se unifica el lenguaje usando solamente JavaScript tanto en el Frontend como en el Backend por lo que es más fácil intercambiar modelo de datos mediante JSON.

3.3 Base de Datos

Para su desarrollo se utilizará un modelo de base de datos no relacional por si en algún momento se quiere añadir más atributos, se puedan añadir fácilmente, en este caso se usará MongoDB.

Por lo que en resumen el stack tecnológico será el siguiente:

- Frontend: React Native + Expo
- Backend: Node.js + Express
- Base de Datos: MongoDB

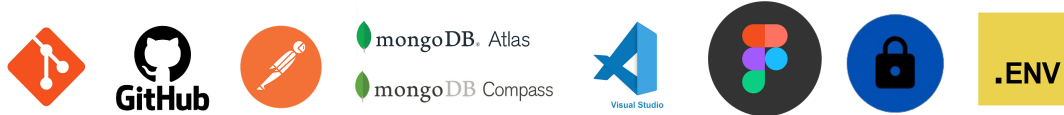


3.4 Herramientas Auxiliares

Para dar soporte al desarrollo y asegurar la calidad del proyecto se han utilizado otras herramientas implementadas en otros proyectos:

1. **Git y GitHub** para control de versiones, que permite tener un historial de cambios, trabajar en diferentes ramas y asegurar de tener el código en la nube como respaldo.
2. **Postman** para pruebas de la API antes de conectar con React Native para probar los endpoints de la API Express, verificando si las rutas devuelven el JSON correcto.
3. **MongoDB Atlas y Compass:**
 - MongoDB Atlas es un servicio en la nube donde se aloja la base de datos real y que permite que la app sea accesible desde cualquier lugar no solo a nivel local.
 - MongoDB Compass es una app de escritorio que permite visualizar los documentos y ver consultas de forma visual.
4. **Visual Studio Code** es el IDE que principal que tiene extensiones para JavaScript y React Native que centraliza todo el desarrollo de la aplicación.
5. **Figma** es una herramienta de diseño de interfaces que será la base del diseño del prototipado de la aplicación, definiendo una línea gráfica con colores, botones, iconos además de la identidad de la aplicación.

6. **Bcrypt** es una librería de cifrado y "haseo" de contraseñas, que al usuario registrarse la librería convierte la contraseña en una cadena de caracteres aleatorios. Por lo que no puede saber las contraseñas ni siendo administrador.
7. **Dotenv** carga variables de configuración desde el archivo externo .env donde se guardará en este proyecto la URL de la base de datos MongoDB Atlas, las claves o la clave de los tokens JWT.



4 Diseño del Sistema

4.1 Arquitectura de la App Cliente-Servidor

En la arquitectura cliente-servidor participan dos componentes: por una parte el cliente que utiliza unos servicios y por otra parte el servidor que ofrece esos servicios. Entre ellos se tiene que establecer una comunicación mediante principalmente internet.

El **rol Cliente** va a ser la parte de la aplicación que inicia las diversas peticiones de servicios al servidor usando la red como medio de comunicación. En el cliente se encuentra toda la funcionalidad que permite al usuario interactuar mediante una interfaz que refleje la idea de la identidad de la aplicación y que sea lo más funcional posible.

El **rol Servidor** es el componente que se encarga de recibir y responder a las peticiones solicitadas desde la aplicación cliente (frontend). En este caso el servidor está desarrollado en Node.js y Express, el que actúa de intermediario con la base de datos. Su función principal es gestionar la comunicación de manera segura con la base de datos MongoDB Atlas y devolver las respuestas correspondientes en formato JSON. Al estar de forma externa del cliente, hace que los datos sensibles y el proceso pesado no afecten al rendimiento del móvil del usuario.

4.2 Diseño de la Base de Datos

Teniendo en cuenta que la aplicación requiere tratar con geolocalización de locales y de reseñas se optó por un diseño de base de datos no relacional utilizando MongoDB. Ya que en comparación a los modelos relacionales, la estructura se organiza en colecciones flexibles de documentos JSON por lo que optimiza las consultas.

El diseño se compone de tres colecciones principales que están conectadas entre sí de manera lógica:

- **Usuarios:** Almacena la información de los usuarios y los registros de los mismos, incluyendo los campos siguientes: (id, nombre, apellidos, foto de perfil, correo, contraseña y lista de locales favoritos).
- **Locales:** Cuenta con todos los locales aptos para celíacos e intolerantes con los siguientes datos: (id, nombre, foto del local, tipo(restaurante, supermercado y cafetería), dirección, ubicación (latitud y longitud), web y nota media del local).
- **Reseñas:** Viculadas a un usuario y a un establecimiento escogido por el usuario en el que consta de los siguientes datos: (id usuario, id local, nombre local, comentario, nota (1 al 5)).

4.3 Diseño de la API REST

Para la comunicación entre la aplicación móvil (cliente) y el servidor se realizará mediante el diseño de una interfaz de programación de aplicaciones basada en la arquitectura API REST. [5] El paso de datos

se realizar bajo el protocolo HTTP/HTTPS utilizando verbos estandarizados para definir las operaciones:

- **POST /api/login:** Para acceder a la aplicación y tras crear un usuario propio a través del registro, se introduce el correo y la contraseña creada y si esos datos son válidos al compararlos con los datos de la base de datos, se accede a la aplicación.
- **POST /api/registro:** El usuario a través de un formulario, rellena con sus datos y se guardan de forma segura creando un nuevo usuario que podrá iniciar sesión luego de que todos los datos estén bien validados.
- **PUT /api/usuarios/actualizar:** Por cada elemento del perfil del usuario va a poder editar, esos datos se guardan y se sobrescriben sobre los originales de la base de datos.
- **DELETE /api/usuarios/eliminar/:id:** Elimina de la base de datos la cuenta del usuario que corresponda el id pasado por parámetro y teniendo que realizar na doble confirmación para borrar.
- **POST /api/locales/crear:** A través de un formulario que rellena el administrador, se cogen esos datos y se va a poder crear un nuevo local que se añadirá a la lista de los demás locales.
- **POST /api/locales/resena:** Un usuario a través de un pequeño formulario que encuentra dentro de las fichas individuales de los locales, esos datos se recogen para crear una nueva reseña que se añade a la lista de comentarios que tiene ese establecimiento.
- **POST /api/locales/favorito:** Dentro de la ficha de un local hay un botón de un corazón donde el usuario al pinchar sobre él. Esta ruta coge el id del usuario y lo añade a la lista de favoritos de ese local. Luego el usuario tiene su propia lista con todos los comentarios que ha creado.
- **GET /api/locales/:id:** Para enseñar una ficha detallada de un local se necesita recuperar un id del local seleccionado para poder mostrar los datos que contiene.
- **GET /api/locales:** Obtener todos los locales para luego enseñarlos al usuario según la cercanía, el tipo de local o el nombre.
- **GET /api/locales/buscar:** Para encontrar un local, el usuario tiene que buscarlo por el nombre del local o por palabras clave que busca por cercanía para luego enseñarlo en la lista del buscador de google maps.
- **GET /api/locales/sugeridos:** Devuelve los 4 locales con mejor calificación como sugerencias en el apartado de favoritos, si está vacío.
- **GET /api/locales/favoritos:** Para enseñar las reseñas de un local se necesita obtener el id de los usuarios para enseñar tanto el comentario como a la nota.
- **POST /api/locales/resena:** EL usuario a través de un formulario que encuentra dentro de cada local en el que tiene que añadir un comentario y del 0 al 5 las estrellas que considere, esos datos se guardan y se pasan a la API que los almacena como nueva reseña.
- **PUT /api/locales/actualizar/:id:** El usuario va a tener un formulario por cada reseña que tenga que le permita modificar su reseña, esos datos se guardan y la api los sustituye por los anteriores datos.
- **DELETE /api/locales/eliminar/:id:** El admin al escoger un local se guarda el id y nos enseña el mismo formulario que para crear local pero con los datos actuales del establecimiento, el admin los puede modificar y actualizar, los nuevos datos de mandan a la api y se sobrescriben sobre los datos anteriores.
- **DELETE /api/resenas/eliminar/:id:** El administrador elimina un comentario que no considera apropiado seleccionando primero el local al que pertenece y mediante doble confirmación la elimina.

- **PUT /api/resenas/actualizar/:id:** Permite al usuario cambiar la información de las reseñas mediante un formulario que permite modificar los datos originales.
- **PUT /api/misResenas:** Se obtiene en la vista de reseñas el historial de comentarios escritos por el usuario incluyendo el nombre del local al que pertenece la reseña.
- **PUT /api/admin/estadisticas:** ofrece unos cálculos al administrador de cuantos locales y usuarios hay registrados.

4.3.1 Seguridad (JWT, validación Zod y bcrypt)

La arquitectura de la API REST se ha diseñado de manera híbrida en la que los endpoints orientados a la consulta de los locales e información general son públicos para optimizar el rendimiento, mientras que los endpoints que manejan los datos sensibles del usuario se encuentran protegidos mediante Tokens JWT. Lo que garantiza al usuario poder acceder a sus propios datos de manera segura sin riesgos a tener filtraciones de ids.

En Kivaa el ciclo de vida del token es el siguiente:

1. **Inicio de Sesión y almacenamiento:** el usuario al introducir su correo y contraseña en la pantalla de Login envía estos datos a la API al verificar que son correctas se genera un Token de acceso (que es una clave única y temporal) y se devuelve a la aplicación. El usuario para que no tenga que introducir la contraseña cada vez que abre la aplicación el token tiene que estar guardado en el móvil. Por lo que se ha integrado la librería "expo-secure-store" que permite almacenar el token en una zona restringida del móvil, que es portegida por el propio sistema operativo.
2. **Comunicación Protegida de la APP con la API:** al tener guardado el token en el dispositivo cada vez que se realice una petición que contenga datos del usuario la aplicación necesita identificarse ante la API. Para no enviar el envío en cada pantalla del Token, con la librería "Axios"
3. **Cierre de Sesión:** El token generado ya no existe cuando el usuario decide cerrar sesión y envía al usuario de nuevo a la pantalla de inicio de sesión. [6]

Listing 1: Generar Token al iniciar sesión el usuario.

```

1  app.post("/api/login", async (req, res) => {
2    try {
3      ...
4      // JWT,generar Token
5      const token = jwt.sign(
6        { id: usuario._id,nombre: usuario.nombre, role: usuario.role, apellidos: usuario.
7          apellidos, clave: usuario.clave},
8        JWT_SECRET,
9        { expiresIn: '2h' }
10     );
11
12     res.json({
13       token,
14       usuario: {
15         id: usuario._id,
16         nombre: usuario.nombre,
17         apellidos: usuario.apellidos,
18         email: usuario.email,
19         role: usuario.role,
20         clave: usuario.clave
21       }
22     });
23   } catch(error) {
24     res.status(500).json({ message: "Error del servidor" });

```

```

25     }
26   });

```

El token generado se va a tener que verificar a través de una función que se ejecuta antes de que la petición llegue a la ruta. Si el token no existe o es inválido, corta la comunicación y devuelve un error.

Listing 2: Middleware para proteger rutas con jwt

```

1  const verificarToken = (req, res, next) =>{
2    //Extrae la cabecera de los headers enviados
3    const authCabecera = req.headers["authorization"];
4    //Formato estandar "Bearer <TOKEN>"
5    const token = authCabecera && authCabecera.split(' ')[1];
6    //Si no hay token, devuelve error
7    if (!token) {
8      return res.status(401).json({ message: "Acceso denegado. No se proporciono un token." });
9    }
10   try{
11     //El token tiene que ser veridico y que no haya caducado
12     const verificado = jwt.verify(token, JWT_SECRET);
13     // Meten los datos decodificados del usuario en el objeto "req"
14     req.usuario = verificado;
15     //Y ahora se va a la ruta
16     next();
17   }
18   catch(error){
19     return res.status(403).json({ message: "Token invalido o caducado." });
20   }
21 }
22

```

Para proteger a los endpoints, por ejemplo el del filtrado de los locales favoritos de un usuario, se inyecta el middleware después de la ruta. Gracias a "req.usuario" se sabe que usuario está pidiendo los datos:

Listing 3: Ejemplo de ruta con middleware

```

1  app.get("/api/locales/favoritos", verificarToken, async (req, res) => {
2    // Al pasar el middleware, req.usuario ya existe y contiene el ID extraido de forma segura
3    const usuarioId = req.usuario.id;
4    const listaFavoritos = await Locales.find({ favoritos: usuarioId });
5    return res.json(listaFavoritos);
6  });
7

```

En el front, se inicia sesión, el token se almacena en el almacenamiento persistente del dispositivo mediante "AsyncStorage". Cuando el usuario navega a una pantalla en la que la petición tenga que ver con información del usuario (como ver los locales favoritos), se extrae el token del almacenamiento local y lo adjunta en los Headers de la petición HTTP utilizando Axios.

Listing 4: Funcionamiento en el Frontend

```

1  useFocusEffect(
2    useCallback(()=>{
3      const datosUsuario = async () => {
4        try {
5          const nombre = await AsyncStorage.getItem("nombreUsuario");
6          const usuarioId = await AsyncStorage.getItem("idUsuario");

```

```

8      const fotoGuardada = await AsyncStorage.getItem("fotoUsuario");
9      // Se recupera el token guardado fisicamente en el dispositivo
10     const token = await AsyncStorage.getItem("token");
11     if(nombre){
12         setNombreUsuario(nombre);
13     }
14     // Si el token existe, configuramos la peticion HTTP
15     if(token){
16         const resultado = await axios.get(`http://10.0.2.2:3000/api/locales/favoritos`,{
17             // Cumplimos con el estandar pasando el token en las cabeceras de autorizacion
18             headers:{
19                 Authorization: `Bearer ${token}`
20             }
21         });
22         // Procesamos los datos devueltos de forma segura por el servidor
23         const idsFavoritos = resultado.data.map((fav: any) => fav._id);
24         setFavoritos(idsFavoritos);
25     }
26     if (fotoGuardada) {
27         setFotoUsuario(fotoGuardada);
28     } else {
29         setFotoUsuario(null);
30     }
31 }
32 catch(error){
33     console.error("Error al cargar los datos", error);
34 }
35 };
36 datosUsuario();
37 }, [])
38 )

```

Para seguir mejorando la seguridad de los datos en la API se implementa un sistema de validación de datos y del cliente (Zod) y el encriptado de las contraseñas de los usuarios (Bcrypt).

La **librería Zod** se encarga en tiempo real de que los campos cumplan los requisitos (por ejemplo: que el correo tenga el formato correo con @, o que los nombres y apellidos tengan una longitud mínima). Si los campos no cumplen con los requisitos la app bloquea la petición HTTP y enseña un mensaje de error al usuario en la aplicación. Esto aporta que el cliente sabe que contenido tiene que insertar y cuando se está equivocando, haciendo la experiencia de usuario más fácil. [7]

Listing 5: Ejemplo de validación de datos al registrarse un nuevo usuario.

```

1      const { z } = require('zod');
2
3
4      const RegistroSchema = z.object({
5          nombre: z.string().min(2, "Nombre demasiado corto"),
6          apellidos: z.string().min(2, "Apellidos obligatorios"),
7          email: z.string().email("Email invalido"),
8          clave: z.string().min(6, "La clave debe tener al menos 6 caracteres"),
9          clave2: z.string()
10     }).refine((data) => data.clave === data.clave2, {
11         message: "Las contraseñas no coinciden",
12         path: ["clave2"],
13     });

```

Al tener los datos ya validados y llegan a la API lo importante es mejorar la seguridad de las contraseñas de los usuarios y no guardarlas en texto plano. El backend utiliza el algoritmo de función has **Bcrypt** para encriptar la contraseña antes de guardarla. Esto aplica un proceso de haséo ubilateral junto con una

técnica de "saltin" que añade caracteres aleatorios a la contraseña. Por lo que significa que:

- Aunque la base de datos sea atacada de forma externa es imposible descifrar las contraseñas de los usuarios.
- Al iniciar sesión, Bcrypt **no descripta la contraseña, sino que coge la que introduce el usuario y aplica el mismo algoritmo matemático y la compara con la que está en la base de datos.** [8]

Listing 6: Hasheo de la contraseña en el registro del usuario.

```
1  const bcrypt = require('bcrypt');
2  //nivel de seguridad
3  const saltRounds = 10;
4
5
6  //Registrarse con los datos y validando que los datos esten correctos
7  app.post("/api/registro", async (req, res) => {
8    try {
9      // Validar con zod
10     const validacion = RegistroSchema.safeParse(req.body);
11     if (!validacion.success) {
12       const erroresFormateados = validacion.error.format();
13       return res.status(400).json({
14         message: "Error de validacion",
15         detalles: erroresFormateados
16       });
17     }
18     const { nombre, apellidos, email, clave } = validacion.data;
19
20     const existeUsuario = await Usuario.findOne({ email });
21     if (existeUsuario) {
22       return res.status(400).json({ message: "El correo ya esta registrado" });
23     }
24
25     //Hashear la contrasena y se guarda en el nuevo usuario todos los datos
26     const passwordHash = await bcrypt.hash(clave, saltRounds);
27     ...
28   });
```

4.4 Creación de la Marca y Prototipado de la Interfaz (UX/UI)

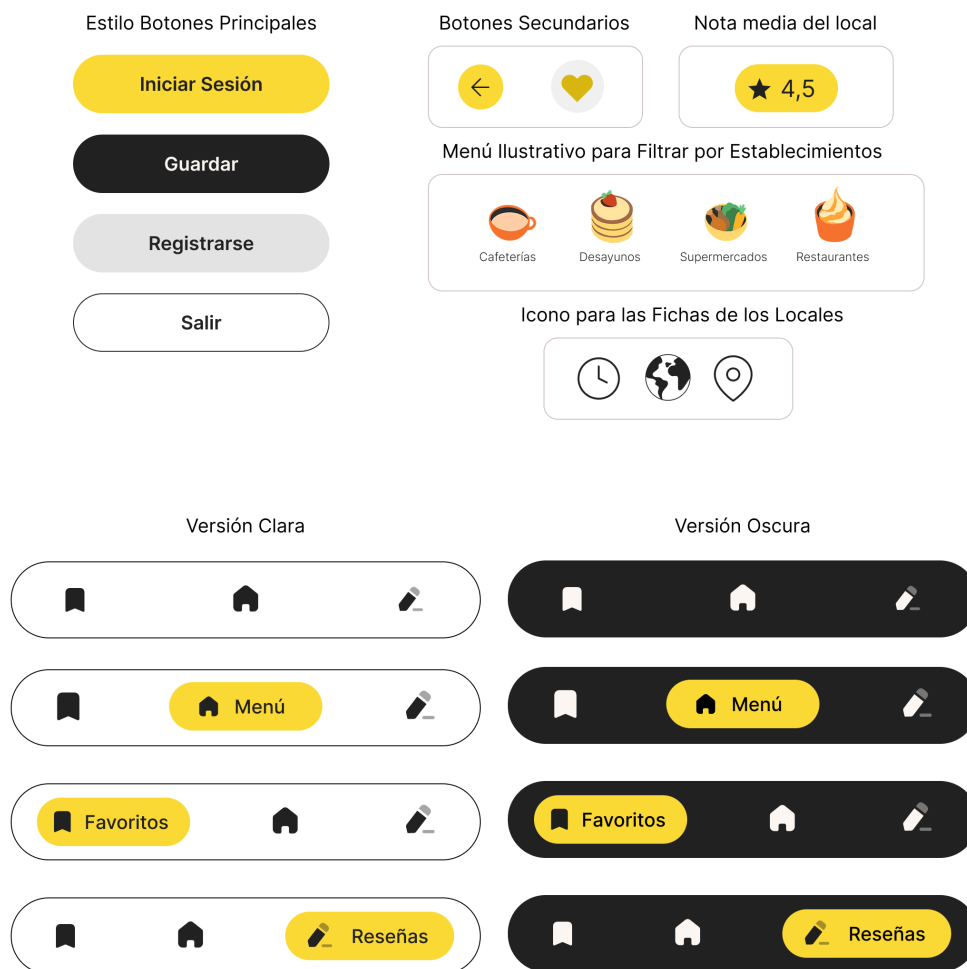
Antes de la realización del prototipado en Figma se ha diseñado una identidad básica de marca que incluye: un **logotipo**, **slogan**, **isotipo**, **tipografías**, **paleta de colores (una primaria y algunos colores secundarios)** y **elementos gráficos** considerados.

Para el nombre se ha elegido Kivaa porque se busca convertir la búsqueda de lugares sin gluten de una experiencia estresante y limitada a una experiencia agradable y fácil. Es una palabra corta que parece inventada pero viene de la palabra **finlandesa "kiva" que significa "agradable", "divertido"**.

Aquí una muestra básica del branding para tener la primera toma de contacto con la aplicación teniendo una base de la marca y estilos.

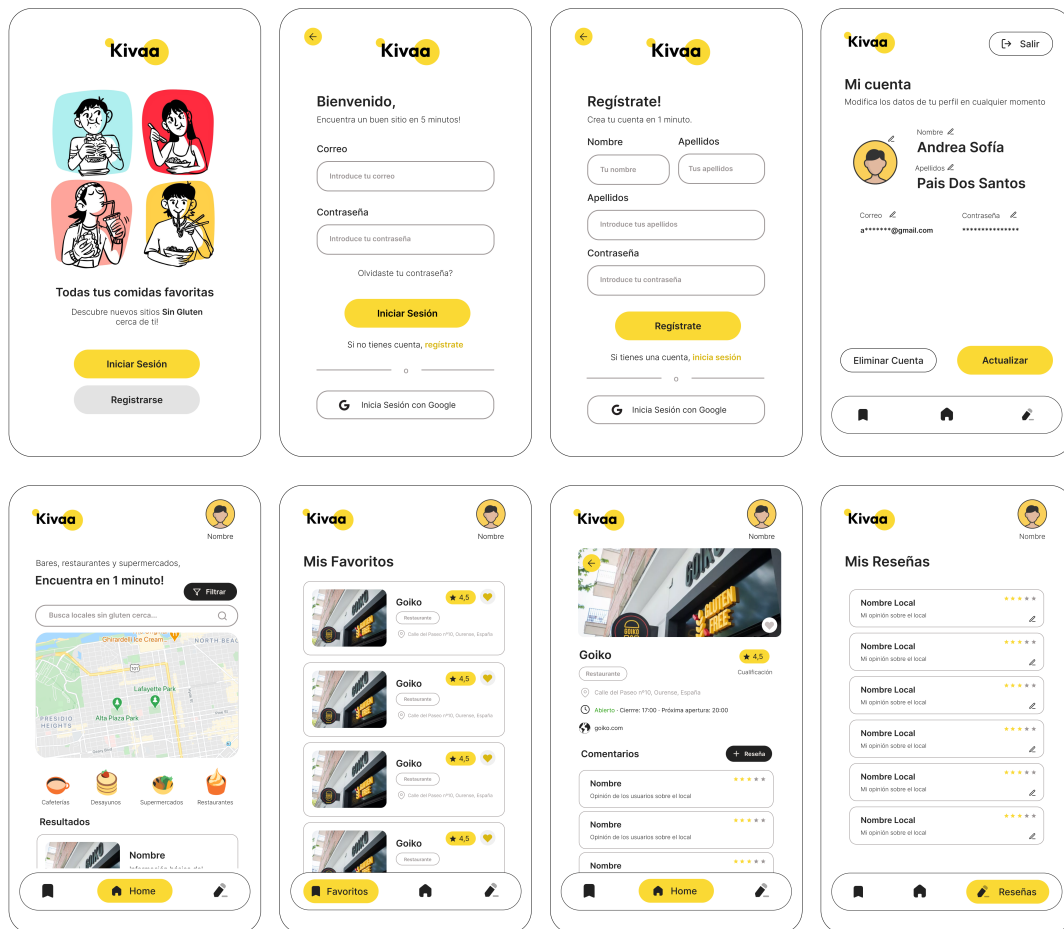


Para reflejar esa estética en la aplicación se diseñaron los primeros elementos visuales que tendrá la aplicación en muchas de sus vistas como sería: **botones, barra de navegación e iconos.**



Al tener la base estética se empezó a desarrollar las vistas principales en Figma. Se realizaron las siguientes propuestas de diseño: **inicio, inicio de sesión, registrarse, página principal, ficha de local, favoritos, mis reseñas y perfil**. Se intento mantener la misma línea estética en todas las vistas y siempre teniendo en cuenta la mejor usabilidad y facilidad para el usuario.

A través del enlace a figma [9] se puede probar el prototipado básico de navegación y ver todas las vistas y elementos de la marca creados. Aquí vemos las vistas y que la línea estética sigue en todas ellas. Se diseñaron las vistas del Usuario principalmente porque las vistas del Administrador son la mayoría formularios básicos por lo que al ya tener el de inicio y registro se basaran en ese estilo por lo que se enfocó en detallar el estilo de la versión del usuario.



Ahora a partir del diseño de las vistas se van a implementar en el frontend de la aplicación.

5 Desarrollo de la Aplicación Móvil

Para la creación de la aplicación Kivaa por un lado hay que construir el lado del cliente que es el frontend y luego implementar la conexión a la base de datos a través de la API REST. El desarrollo se ha guiado bajo la reutilización de código, modularidad y optimización a través del framework React Native y las herramientas de despliegue de Expo.

5.1 Implementación del Frontend

El Frontend se divide en dos bloques según el rol del usuario: por un lado el cliente y por otro el administrador que gestiona el contenido que tendrá acceso el usuario. Antes de acceder a cada rol hay que **iniciar sesión** con el correo y contraseñas o **registrarse** con nombre, apellidos, correo y contraseña repetida 2 veces. Al registrar, el usuario hay que iniciar sesión otra vez.

Con el rol de usuario se tendrá acceso:

- **Pantalla Principal** en la que podrá buscar los locales e interactuar con ellos.
- **Pantalla de Locales Favoritos** almacena en una lista los establecimientos el cual el usuario haya interactuado con ellos dándole al icono del corazón.
- **Pantalla Ficha de Local** a través de la pantalla principal o de la de los favoritos se accede a la ficha del local con sus datos detallados en el que se pinche.
- **Pantalla de Reseñas** almacena la lista de comentarios que crea ese usuario de todos los locales y permite editar esos comentarios en cualquier momento.
- **Pantalla de Datos del Usuario** le permite actualizar los datos y cerrar sesión cuando quiera.

El lado del administrador cuenta con las siguientes vistas:

- **Pantalla Principal** tiene unas estadísticas básicas de cuantos locales hay en total y cuantos usuarios hay. Luego cuenta con un menú con varios apartados:

Crear un Nuevo Local se crea a través de un formulario que se encuentra en un modal al seleccionar la opción de "Añadir local".

Editar un Local se accede a través de la **Pantalla de Locales** donde primero buscamos el local que queremos modificar. El administrador le da la icono de editar y le abre el mismo modal que el de crear local pero con los datos ya introducidos para cambiarlos y actualizarlos si se necesita.

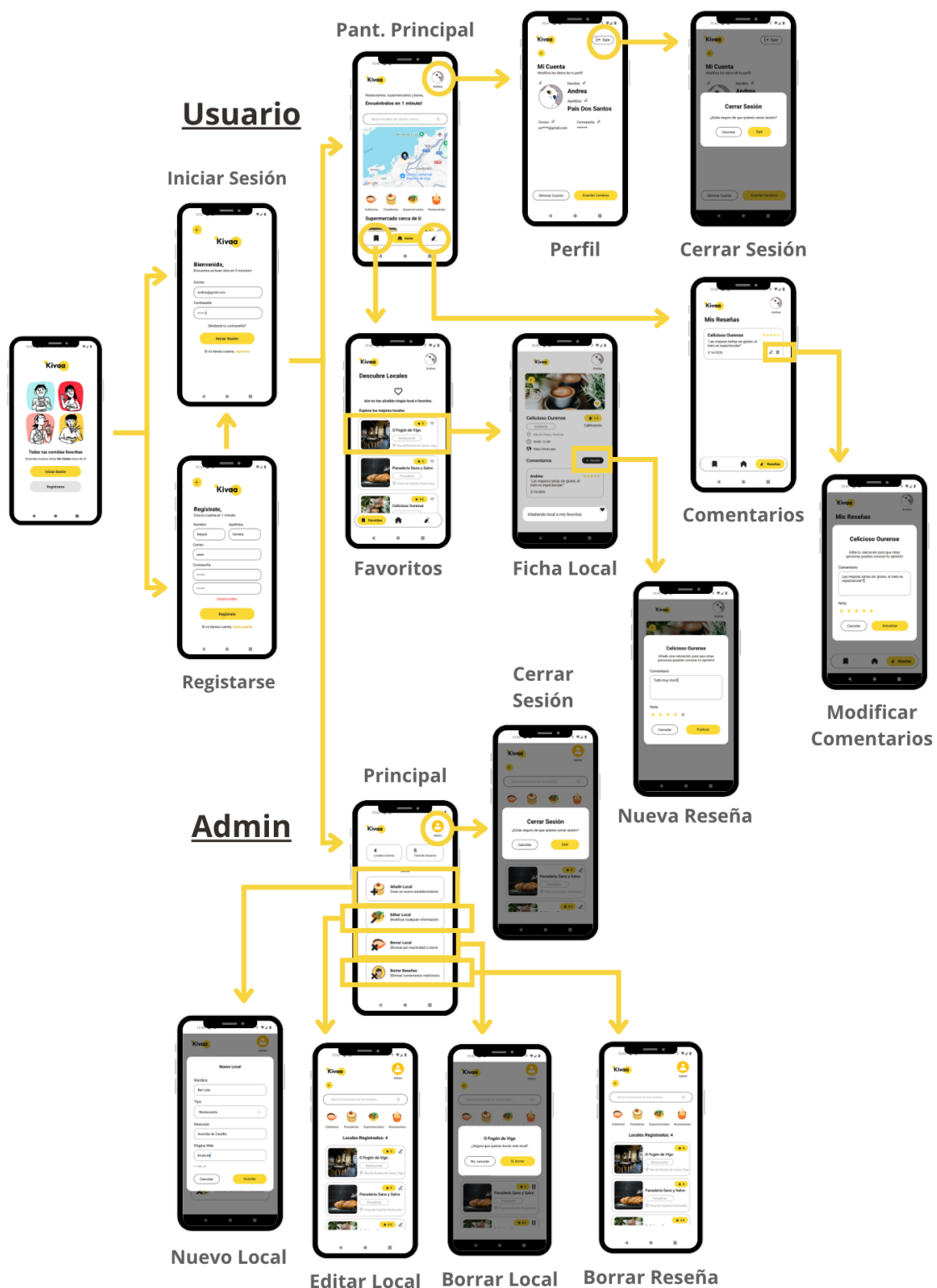
Borrar un Local se accede también a través de la **Pantalla de Locales** donde se escoge el local y al darle el icono de borrar, tiene que volver a confirmar para eliminarlo.

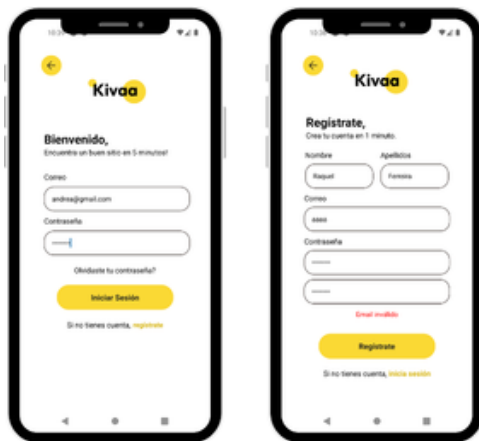
Borrar Reseñas tiene que navegar a la **Pantalla de Locales** seleccionar el local y dentro la ficha del local va a ver todos los comentarios. El administrador selecciona el comentario y el icono de borrar tiene que volver a confirmar para eliminarlo.

- **Pantalla de Locales** es el puente entre las opciones y las funcionalidades de que se busca. Se muestra el listado de locales y a través del buscador también se pueden buscar.

Para una fácil visualización de la navegación entre vistas se representa mediante un mapa con las diferentes pantallas para tener una idea general de la arquitectura de la aplicación. Además de que sea lo más sencillo posible para el usuario.

Por cada rol de usuario, el diseño y la funcionalidad va acorde a lo que necesita donde todas las vistas intentan mantener una estética sencilla y contrastada.





Iniciar Sesión

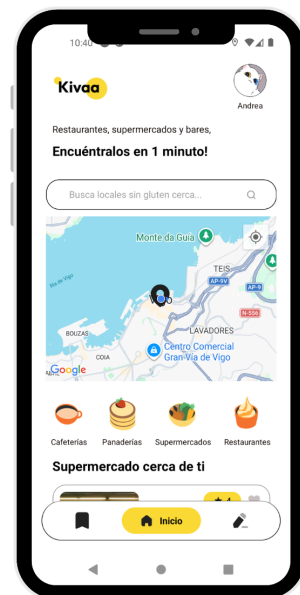
Regístrate

Las primeras pantallas constan de unos formularios básicos que tienen en cuenta el tipo de dato de cada input. Por ejemplo, para el correo, el teclado cambia e implementa la utilización de "@". Otro ejemplo sería para los input de contraseña que las oculta para mayor seguridad.

Por otro lado, en el backend se **validan todos los input con Zod**, ya mencionado anteriormente, que nos avisa de cualquier tipo de error en los campos mandando un mensaje de error al front, denegando la petición, esperando a que el usuario rellene los campos correctamente para volver a lanzarla.

La pantalla principal reúne más elementos gráficos para ir guiando al usuario desde el principio ofreciéndole diferentes caminos. Por un lado tenemos **un buscador**, permite al usuario escribir tanto el nombre del local, como nombre de las categorías y buscar por cercanías.

- **El buscador** imprime los resultados en una lista flotante que al pinchar en una opción el mapa te lleva a la ubicación del local. Y también imprime los resultados en una lista abajo de todo para una vista general del local.
- **El menú de iconos** consta de las 4 categorías a las que pueden pertenecer los locales: (Cafetería, Panadería, Supermercado y Restaurante). Cada categoría muestra una lista de los locales de ese tipo y mostrando si está cerca o lejos de ti.
- **Las listas** se componen de tarjetas con el mismo diseño que se imprime en función de cuantos locales tiene cada lista.
- **El mapa** se encarga de mostrar visualmente donde se ubica cada local de se busque.
- **Barra de navegación** entre vistas solo es visible después de iniciar sesión. Esta barra contiene la pantalla de inicio, junto a la pantalla de favoritos y la de reseñas, para que al usuario le sea más cómodo a la hora de navegar entre ellas.



Pant. Principal

Es importante mencionar como ha sido el proceso de la **implementación de mapas y geolocalización** utilizando la librería "react-native-maps". Permite renderizar un mapa interactivo que se comunica con la API REST y la base de datos de MongoDB. La arquitectura de búsqueda se divide en:

- **Almacenar las Coordenadas en el Backend:** en el modelo de datos de la base de datos, cada local tiene definidos obligatoriamente los campos "latitud" y "longitud" de tipo (Number).
- **Permisos y centrado de la posición del usuario:** al iniciar la pantalla principal, la aplicación solicita al usuario permisos de ubicación. Si el acceso es concedido, el sistema coge las coordenadas GPS del dispositivo (latitud y longitud) y actualizar el estado de la región del mapa (setRegion) moviendo la cámara del mapa a la ubicación real del usuario. Y para marcar el punto azul en el mapa de ubicación se usa (showsUserLocation=true).

Listing 7: Permisos y ubicación del usuario.

```

1
2  useEffect(() => {
3      const datosIniciales = async () => {
4          // Pedir permiso de ubicacion al dispositivo movil
5          let { status } = await Location.requestForegroundPermissionsAsync();
6          if (status !== "granted") {
7              console.log("Permiso denegado");
8              return;
9          }
10         // Obtener la ubicacion GPS actual
11         let ubicacion = await Location.getCurrentPositionAsync({});
12
13         // Actualizar la region del mapa con tu posicion real
14         setRegion({
15             latitude: ubicacion.coords.latitude,
16             longitude: ubicacion.coords.longitude,
17             latitudeDelta: 0.05,
18             longitudeDelta: 0.05,
19         });
20
21         // Carga de locales desde la API REST
22         try {
23             const res = await axios.get("http://10.0.2.2:3000/api/locales");
24             setLocales(res.data);
25         } catch (error) {
26             console.error("Error cargando locales:", error);
27         }
28     }
29     datosIniciales();
30 }, []);

```

- **Renderizado Dinámico de Marcadores:** en el frontend se realiza una petición para obtener los locales y mediante un método iterativo ".map" dibuja un marcador sobre el mapa por cada establecimiento. Posicionando exactamente las coordenadas reales y luego tener una ventana emergente al pulsar sobre ellos con el nombre y el tipo del local.

Listing 8: Permisos y ubicación del usuario.

```

1
2  <MapView
3      style={styles.mapa}
4      ref={mapRef}
5      region={region}
6      onRegionChangeComplete={setRegion}
7      provider="google"
8      showsUserLocation={true}
9  >
10     {/* Mapeo de locales para dibujar los pines en sus coordenadas */}
11     {localesBusqueda.map((local) => (
12         <Marker
13             key={local._id}
14             coordinate={{
15                 latitude: local.latitud,
16                 longitude: local.longitud,
17             }}
18             image={require('@assets/images/PIN.png')}
19             onPress={() => {
20                 setLocalSeleccionado(local);
21                 setModalLocal(true); // Abre el modal de la tarjeta del local
22             }}
23         />

```

```

24     }}
25 </MapView>

```

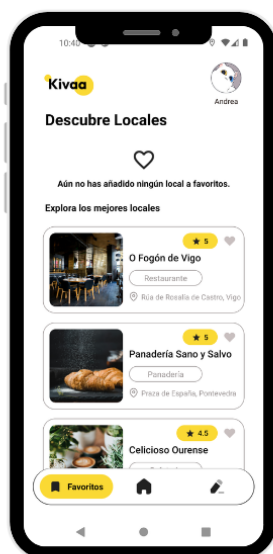
- **Posición de la cámara con el buscador y calcular locales cercanos:** usando (`mapRef.current`) y (`animateToRegion()`) el usuario al escribir el nombre de un local en el buscador, escribiendo una categoría o buscando cercanía, la cámara se desplaza automáticamente hacia las coordenadas de destino haciendo un poco de zoom al marcador. También se implementó la **fórmula de Haversine** para calcular los kilómetros reales en línea recta entre dos puntos por lo que así sabemos que locales están cerca de la ubicación del usuario.

Listing 9: Localización y búsqueda locales cercanos.

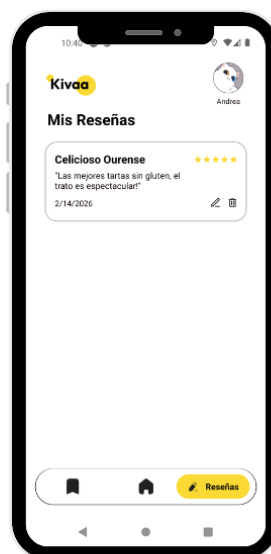
```

1
2  const irLocal = (lat: number, lng: number) => {
3      mapRef.current?.animateToRegion({
4          latitude: lat,
5          longitude: lng,
6          latitudeDelta: 0.01, // Mas zoom sobre el local elegido
7          longitudeDelta: 0.01,
8          }, 1000) // Duracion de la animacion
9  };
10
11  // Implementacion de la formula matematica para calcular distancias geograficas
12  const calcularDistancia = (lat1: number, lon1: number, lat2: number, lon2: number) => {
13      const radio = 6371; // Radio de la Tierra en kilometros
14      const dLat = (lat2 - lat1) * (Math.PI / 180);
15      const dLon = (lon2 - lon1) * (Math.PI / 180);
16      const a = Math.sin(dLat/2) * Math.sin(dLat/2) +
17          Math.cos(lat1*(Math.PI/180)) * Math.cos(lat2*(Math.PI/180)) * Math.sin(
18              dLon/2) * Math.sin(dLon/2);
19      const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
20      return radio * c; // Devuelve la distancia exacta en kilometros
21  }

```



Favoritos



Comentarios



Modificar
Comentarios

Las vistas de **Mis Favoritos** y **Mis Reseñas** comparten una misma arquitectura técnica y lógica. Solicitan datos mediante endpoints de la API REST protegidos mediante autenticación (token) gestionando estados y renderizando listas dinámicas. Los puntos más importantes de estas vistas son:

- **Pantallas siempre actualizadas** mediante una función que detecta cada vez que la pantalla se pone en primer plano, volviendo a cargar los datos por si hubo modificaciones.

Listing 10: Ejemplo de recarga de datos en la vista Reseñas.

```

1
2  useFocusEffect(
3    useCallback(() => {
4      const obtenerDatos = async () => {
5        try {
6          const token = await AsyncStorage.getItem("token");
7          if(token){
8            // Pide al servidor las comentarios actualizadas del usuario
9            const resultado = await axios.get(`http://10.0.2.2:3000/api/misResenas`, {
10              headers: { Authorization: `Bearer ${token}` }
11            });
12            setComentarios(resultado.data); // Guarda los comentarios en la pantalla
13          }
14        } catch(error) {
15          console.error("Error al cargar los datos", error);
16        }
17      };
18      obtenerDatos();
19    }, [])
20  );

```

- **Gestión de listas vacías**, para enseñar los locales y comentarios se utiliza el componente "FlatList" que se ha configurado que para cuando no haya datos enseñe al usuario un mensaje o que por ejemplo enseñe una lista de locales sugeridos por mayor nota.

Listing 11: Ejemplo de recarga de datos en la vista Reseñas.

```

1
2  <FlatList
3    data={favoritos} // Lista de favoritos
4    keyExtractor={(item) => item._id}
5    ListEmptyComponent={
6      // Esto es lo que se dibuja si la lista de favoritos esta vacia
7      <View style={styles.contenedorOpcion}>
8        <Image source={require('@assets/images/favorite.png')} style={styles.iconoCorazon}
9        />
10       <Text style={styles.textoBold}>Aun no has metido ningun local a favoritos.</Text>
11     </View>
12   }
13   renderItem={({ item }) => (
14     // Esto dibuja la tarjeta del local si la lista si tiene datos
15     <TouchableOpacity style={styles.tarjeta} onPress={() => router.push({ pathname: "/"
16       PantallaLocal", params: { id: item._id } })}>
17     /* Contenido de la tarjeta */
18     </TouchableOpacity>
19   )}
20 />

```

- **Ventanas emergentes y avisos temporales** para ayudar al usuario se implementan "Modales". Por ejemplo para editar una reseña no hace falta cambiarse de vista, se abre una ventana emergente que permite editar el contenido y al guardar sale otro modal temporal informando si la operación se ha realizado con éxito o no.

El resto de vistas utilizan las mismas herramientas: FlatList, Modals, ScrollView, etc. La única diferencia es que se realizan peticiones diferentes dependiendo de lo que necesita el usuario pero la funcionalidad sigue siendo la misma.

5.2 Integración con la API y Base de Datos

Para que la aplicación funcione en tiempo real, se comunica con un servidor y una base de datos en la nube mediante los siguientes mecanismos:

- **Librerías de Conexión (Axios):** para la comunicación entre la aplicación móvil (React Native) y el servidor se realiza mediante la librería Axios. Se utiliza como mensajero encargado de enviar las peticiones del usuario y recibir las respuestas del servidor. Se utilizó esta librería porque automatiza tareas repetitivas y la gestión de errores.
- **Persistencia Local:** está pensada para trabajar con conexión a internet para garantizar que los datos estén actualizados siempre. Se utiliza el sistema de almacenamiento interno del teléfono (AsyncStorage) para guardar información como sesión iniciada para que no tenga que iniciar sesión cada vez que se abra la app. Y si el usuario se queda sin internet, Axios detecta el fallo de red. En lugar de cerrarse con un error, la aplicación lo gestiona de forma segura: mantiene al usuario dentro de la app gracias a los datos guardados en el teléfono y muestra un aviso visual intuitivo indicándole que necesita conectarse a la red para cargar la información
- **Integración (Base de Datos):** para enseñar como el servidor gestiona y organiza la información que recibe desde la aplicación móvil, se enseña el diseño de los modelos de datos utilizados en la base de datos:

Listing 12: Estructura para almacenar los locales en la Base de Datos.

```
1  const localesEsquema = new mongoose.Schema({
2    nombre: { type: String, required: true },
3    tipo: { type: String, required: true, enum: ["Restaurante", "Cafeteria", "Supermercado",
4      "Panaderia"] },
5    direccion: { type: String, required: true },
6    latitud: { type: Number, required: true },
7    longitud: { type: Number, required: true },
8    calificacion: { type: Number, default: 0 },
9    horario: { type: String, required: true },
10   enlace: { type: String },
11   foto: { type: String },
12   // Lista de IDs de los usuarios que han marcado este local como favorito
13   favoritos: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Usuario' }]
14 });
15
16 // Estructura para almacenar los comentarios de los usuarios
17 const resenaEsquema = new mongoose.Schema({
18   localId: { type: mongoose.Schema.Types.ObjectId, ref: 'Locales', required: true },
19   usuarioId: { type: mongoose.Schema.Types.ObjectId, ref: 'Usuario', required: true },
20   usuarioNombre: { type: String, required: true },
21   comentario: { type: String, required: true },
22   estrellas: { type: Number, required: true, min: 1, max: 5 },
23   fecha: { type: Date, default: Date.now }
24 });
```

6 Estudio de Viabilidad y Utilización Empresarial

Tras desarrollar la aplicación se describe la inversión necesaria para que sea un producto mínimo viable (MVP). Hay que tener en cuenta los costes de personal, la inversión tecnológica y licencias necesarias.

1. **Costes de Personal:** en este caso la mayor parte del coste es el tiempo de desarrollo al ser un trabajo de fin de ciclo pero para calcular la viabilidad se aplica tarifas de mercado teniendo en cuenta que sería partiendo de un desarrollador de grado superior (Junior). Por lo que aproximadamente se han dedicado **300 horas distribuidas en 4 meses**. Por lo que por esta parte serían unos **6.100€**.
2. **Inversión en Tecnología:** la ventaja es que el stack tecnológico escogido (Node.js, Expo y MongoDB) es que permite iniciar el proyecto con sus versiones gratuitas.
3. **Licencias:** para mantener el hosting del backend serían unos **7€ al mes** unos **84€ al año**. La licencia de Google Play se realiza un pago único de **25€** y la posibilidad de subirlo a Apple Store siendo unos **99€ al año**.
4. **Costes de Hardware:** pensando en un equipo informático de gama media para el desarrollo de dispositivos físicos para las pruebas reales, por lo que por un lado el equipo en el que se desarrolla tiene un valor estimado de 1200€ el coste de amortización por unos 3 meses serían de **75€**.

Por lo que el coste total del primer año para el desarrollo y mantenimiento del proyecto es de: **6.383 €**. El proyecto es muy viable al apoyarse en tecnologías de código abierto y servicios en la nube. A parte Google al implementar en el proyecto Google Maps, la aplicación para que empiece da unos 200 euros de crédito cada mes.

El proyecto no puede ser viable solo porque sea útil, **tiene que ser viable como producto** y se encontraron las siguientes vías de explotación:

- **Modelo Gratuito y Suscripción:** la utilización básica de la aplicación sería gratuita, se puede implementar un modelo de suscripción mensual o anual muy baja para desbloquear (**Sin publicidad** (eliminando los patrocinios de marcas), **modo Offline** (para descargar el mapa para viajar sin preocuparse por tener internet o no) y alguna posibilidad de **Contenido exclusivo** (como descuentos en locales)).
- **Patrocinios y Publicidad:** al ser una app centrada en un público objetivo tan pequeño las marcas interesadas sería alimentarias pudiendo aparecer: **Publicidad de marcas** (a través de anuncios específicos) y **destacar locales** (que esos locales paguen una cantidad pequeña al mes para salir entre los primeros resultados en las búsquedas).
- **Acuerdos con Asociaciones:** por el ejemplo con el FACE (Federación de Asociaciones de Celíacos de España) donde los de la asociación podrían usar el panel de administrador para **validar los datos de los locales** y **generar informes de datos sobre la necesidad de más demanda de locales por zonas**.

7 Pruebas y Validación

7.1 Pruebas de Usuario

Estas pruebas demuestran que la app es fácil de usar y cumple los requisitos del cliente. Todos los pasos se tienen en cuenta desde el punto de vista del usuario mediante los botones, textos, modales, etc.

ID	Acción	Lo que hace el Usuario	Resultado Esperado	Resultado Real	Estado
PU-01	Registro de Usuario	Introduce los datos válidos y pulsa "Regístrate"	El usuario se crea y se guarda en la Base de Datos y lleva a iniciar Sesión.	Funciona correctamente	Correcto
PU-02	Registro Erróneo	Introduce email sin "@"	Zod frena la petición y muestra en pantalla un mensaje de error en rojo.	Se tiene que introducir el dato correcto para registrarse	Correcto
PU-03	Inicio de Sesión	Introduce los datos válidos, al pulsar "Iniciar Sesión" lo lleva a la pantalla principal guardando sus datos.	Los datos del usuario se guardan y lleva al usuario a la pantalla principal	Funciona correctamente	Correcto
PU-04	Buscador de Locales	Escribe "algo" en el buscador	Enseña la lista según lo buscado (tipo,nombre del local o cercanía).	Filtra los locales según lo esperado	Correcto
PU-05	Buscador de Locales	Al buscar "cualquier cosa" si no encuentra coincidencias enseñar al usuario un mensaje de "No se ha encontrado locales que coincidan con la búsqueda"	Buscar en la BD y al no encontrar coincidencias enseñar un mensaje	No aparecía el mensaje tuve que corregirlo, pero luego aparecía	Corregido
PU-06	Vista Detallada de Local	Pulsa sobre la tarjetade un local específico de la lista o del mapa	La app navega a una nueva vista pasando el ID del local y enseña toda su información	Carga la información del local seleccionado	Correcto
PU-07	Añadir y ver locales Favoritos	Pulsa en el corazón de la tarjeta del local que interese	Al dar a favorito salta una alerta de "añadido a favoritos". El usuario tiene un apartado donde se visualiza.	Se guarda el id del local y se añade a la lista personal de favoritos, se puede enseñar y editar	Correcto
PU-08	Añadir una Reseña	Dentro de la ficha del local, el usuario pulsa en "+ Reseña" se abre un formulario que pide nota y comentario y le da al botón de guardar	Se guarda la información del formulario y se añade una reseña	Se añade a las reseñas del local y en el apartado de reseñas lista las reseñas escritas por ese usuario	Correcto
PU-09	Editar Datos Perfil	En el perfil le da a editar y le abre un formulario con el dato actual y al cambiarlo y al guardarlo se cambia automáticamente en la ficha principal	Cambio automático de los datos si el contenido del formulario es diferente al original	Al tener el id del usuario, el mismo puede editar sus datos facilmente en el menú y al guardar cambiarlo en la base de datos	Correcto
PU-10	Cerrar Sesión	En el perfil le da a salir, sale un modal que al confirmar redirige a la página de inicio	Cierre de sesión a través de doble confirmación no deja acceder a ningún tipo de contenido	Al cerrar sesión, el token expira y hay que volver a iniciar sesión	Correcto

Tras la ejecución de las pruebas unitarias y de integración por parte de los usuarios, se concluye que la aplicación funciona correctamente, se mantiene estable y alienada a los objetivos principales del proyecto, habiendo corregidos pequeños errores durante la fase de pruebas con usuarios.

8 Escalabilidad de Kivaa

Para el correcto crecimiento de la aplicación y mejorar la experiencia de los usuarios se han planificado las siguientes mejoras para próximas versiones:

- **Implementación de Modo Oscuro:** para mejorar la accesibilidad y reducir la fatiga visual, brindando una interfaz más moderna y dar la posibilidad al usuario de escoger la que más le interese.
- **Inicio de Sesión Rápido con Google:** para simplificar el proceso de registro e inicio de sesión donde los usuarios pondrán acceder a la aplicación con un solo toque sin necesidad de rellenar formularios ni recordar contraseñas.
- **Sistema para Compartir y Recomendar Locales:** implementar una función social que permita a los usuarios recomendar locales a otros usuarios a través de la aplicación o a través de otras redes sociales como WhatsApp y poder enviar mensajes entre usuarios.
- **Añadir notificaciones Personalizadas:** para enviar alertas directamente a la pantalla de bloqueo de teléfono del usuario, por ejemplo si un local cambia su hora de apertura o de cierre, o si le comparte algún local otro usuario, etc.
- **Sistema de Logros y Niveles:** el usuario podría ganar puntos, medallas o niveles por ejemplo por todos los comentarios hechos en locales, por compartir locales a usuarios, etc.

9 Manual de Instalación y Configuración

Antes de iniciar la instalación del proyecto hay que tener instaladas las siguientes herramientas globales:

- **Node.js:** Versión LTS
- **NPM:** Gestor de paquetes integrados con Node.js
- **Git:** Para el control de versiones y la clonación del proyecto
- **Expo Go:** Que no es obligatoria, es por si se quiere hacer las pruebas en un dispositivo físico, en este proyecto se usó para pruebas un emulador de Android Studio

Por un lado tenemos que **configurar la base de datos** en este caso MongoDB. Se puede optar por una instalación local o en la nube.

- **MongoDB Atlas (Nube)**

Hay que registrarse en MongoDB Atlas.

Crear un nuevo Cluster gratuito.

En la sección **Database Access** crear un usuario y contraseña.

En **Network Acces** hay que añadir la dirección IP 0.0.0.0/0 para permitir conexiones desde cualquier entorno.

Copiar la cadena de conexión que genera Atlas y pegarla en el archivo .env.

- **MongoDB Local**

Descargar e instalar **MongoDB Community Server**.

El servicio esté corriendo en el puerto por defecto en este caso: "mongodb://localhost:27017/nombreBD".

Tras configurar la base de datos, clonamos el proyecto del primer repositorio (lo tengo separado en 2 repositorios: el primero contiene el server y la documentación y el otro todo el front). Navegamos a la carpeta donde está el servidor y ejecutamos en la terminal la instalación de las dependencias:

Listing 13: Comando para instalación de dependencias

```
1 npm install
```

Luego creamos las **variables de entorno** en un archivo .env que va a tener que contener lo siguiente:

Listing 14: Variables de entorno necesarias para este proyecto

```
1 MONGO_DB=mongodb://localhost:27017/nombreDB
2 PORT=3000
3 JWT_SECRET=clave_para_tokens
```

Para el **despliegue del servidor** iniciando la API REST con el siguiente comando:

Listing 15: Comando para inicializar la API REST

```
1 node ./server.js
```

Por otro lado tenemos la **configuración del Frontend** (React Native con Expo). Primero clonamos el proyecto que está en el segundo repositorio y ejecutamos al estar en de la aplicación el siguiente comando para las dependencias:

Listing 16: Comando para instalación de dependencias

```
1 npm install
```

Hay que tener en cuenta que si se intenta hacer pruebas con un dispositivo físico hay que **configurar la conexión de Red**. No se podría usar localhost ni 127.0.0.1 en las peticiones HTTP del frontend porque el móvil no reconocería el servidor del ordenador.

Por lo que hay que obtener la **ip del ordenador** y en el archivo .env apuntar a esa IP:

Listing 17: Configuración para dispositivos de pruebas físicos

```
1 export const API_URL = "http://192.168.1";
```

Para ejecutar la aplicación usamos el siguiente comando:

Listing 18: Comando de ejecución de Frontend

```
1 npx expo start
```

En mi caso, al usar un emulador de android tengo que abrirlo primero, y tras ejecutar la app darle a la "a" para que lo ejecute en el emulador que tengo abierto. Si se quiere probar en dispositivos físicos se tendría que tener la aplicación Expo Go y escanear el código que aparece por terminal.

10 Conclusiones

Realizar este proyecto ha sido interesante porque a diferencia de otros trabajos donde las pautas y la temática están marcadas, plantear una idea propia desde cero, diseñarla desde el principio y verla convertida en una aplicación real fue un proceso largo pero gratificante.

Al acabar el trabajo he aprendido que no es lo mismo la idea que tenemos al principio a lo que acabamos realizando al final. Hay que ser realistas desde un principio ya que hay muchos diseños que al intentar llevarlos a la realidad nos traba y a veces con diseños más simples se consigue una experiencia de usuario mejor que con un diseño más cargado. También me he peleado bastante al intentar arreglar errores, ya que al empezar a tener muchas vistas era más difícil solucionar los errores simples.

Desde el principio del proyecto he intentado ponerme en la piel del usuario y no pensar que la cantidad está por encima de la calidad. Buscaba que fuera cómoda de usar, fácil y amigable, por ejemplo mediante alertas que visualmente se vean agradables e intentar no marear al usuario con demasiadas pantallas.

Este proyecto nació con la idea de cubrir una necesidad real, ayudar a las personas a encontrar locales de confianza de forma sencilla y segura. Ver que conseguí crear todo lo que tenía en mente me hace sentir orgullosa del trabajo realizado.



11 Bibliografía

Fuentes consultadas

- [1] Celicidad (s.f.). *Las cifras de la celiacía*.

<https://celicidad.net/las-cifras-la-celiaquia/#::~text=%2DEntre%20un%201%20y%20un,los%20celiacos%20no%20est%C3%A1n%20diagnosticados.>

- [2] Urian Viera. (s.f.). *Introducción a React Native: Crea Apps Móviles Nativas en iOS y Android*. Web Developer.

<https://urianviera.com/react-native/introduccion-a-react-native>

- [3] Reboot. (2026, 23 de mayo). *Introducción al enrutador Expo*. Expo Documents.

<https://docs.expo.dev/router/introduction/>

- [4] GitBooks. (s.f.). *Creación de una API con node.js*.

https://juanda.gitbooks.io/webapps/content/api/creacion_de_una_api_con_nodejs.html

- [5] Castillo Lacruz, L.J. (2023, 14 de agosto). *Construyendo una API con Node + Express + MongoDB*. Medium.

<https://ljcl79.medium.com/construyendo-una-api-con-node-express-mongodb-f5c9ec86b908>

- [6] López Magaña, L.M. (2020, 17 de enero). *Qué es Json Web Token y cómo funciona*. OpenWebinars.

<https://openwebinars.net/blog/que-es-json-web-token-y-como-funciona/>

- [7] Gao, J. (2026, 21 de marzo). *Creación de una API REST con tipado seguro mediante Zod, Express y TypeScript (Guía 2026)*. Dev Community.

https://dev.to/young_gao/building-a-type-safe-rest-api-with-zod-express-and-typescript-1

- [8] NPM. *node.bcrypt.js*.

<https://www.npmjs.com/package/bcrypt>

- [9] Pais Dos Santos, A. (s.f.). *Diseño de Vistas e Identidad*. Figma.

<https://www.figma.com/design/G1i1CM2kr6SKkXYXdm5spK/IdeaTFC?node-id=60-478&t=raRFfJIM19F4k75G-1>

- [10] Reboot. (s.f.). *React Native Maps: Cómo instalar y usar la librería de mapas en iOS y Android [2020]*.

<https://reboot.studio/blog/es/react-native-maps-2020>